

Documentação Técnica

Arquitectura, convenções e operação

Como o sistema está construído: camadas, isolamento entre organizações, autorização, integração entre módulos, base de dados, deployment e as armadilhas que já custaram tempo a alguém.

Versão

Setembro de 2026

Destinatários

Equipa de desenvolvimento e operação

Stack

Node.js · Express · Prisma · PostgreSQL · Next.js

Índice

1. Panorâmica

- Stack e componentes
 - Estrutura do repositório
-

2. Arquitectura do backend

- Camadas e responsabilidades
 - Composição das rotas
 - Serviços base
-

3. Isolamento entre organizações

- O contexto de organização
 - Onde o isolamento se aplica
-

4. Autenticação

- Tokens e validade
 - Renovação no cliente
-

5. Autorização

- As quatro barreiras
 - RBAC e perfis
 - Códigos de erro
-

6. Modelo de dados

- Domínios e entidades
 - Enumerações
-

7. Integração entre módulos

- Event bus
 - Reacções e idempotência
 - Cold start em serverless
-

8. Módulo de Contabilidade

- Plano de contas e sincronização
 - Lançamentos automáticos
 - Ordenação de códigos
-

9. Convenções da API

- Formato das respostas
 - Paginação e erros
 - Documentação Swagger
-

10. Arquitectura do frontend

- Grupos de rotas e layouts
 - Camadas service, hook e DTO
 - Estado e cache
-

11. Validação

12. Base de dados e migrações

- Prisma e adaptador
 - Migrações de dados
 - Seeds
-

13. Deployment e ambiente

- Projectos e variáveis
 - Tarefas agendadas
 - Ordem de deploy
-

14. Testes

15. Armadilhas conhecidas

16. Runbooks

1. Panorâmica

O Mpamba é um ERP modular multi-organização. Duas aplicações independentes, uma base de dados PostgreSQL, e quatro módulos de negócio que comunicam por eventos internos.

1.1 Stack e componentes

COMPONENTE	TECNOLOGIA	PAPEL
<code>mpamba-backend</code>	Node.js · Express 5 · TypeScript	API REST, regras de negócio, autorização.
<code>mpamba-frontend</code>	Next.js 16 · React 19 · TypeScript	Aplicação web (App Router).
Persistência	PostgreSQL · Prisma 7	45 modelos, 26 migrações versionadas.
Cache	Redis	Cache de respostas, via <code>cache.middleware.ts</code> .
Validação	Zod 4	DTOs partilhados em forma nos dois lados.
Estado no cliente	TanStack Query 5 · Zustand	Cache de servidor e sessão.
Testes	Vitest 4	28 ficheiros, 257 testes.
Alojamento	Vercel (serverless)	Projectos separados para API e web.

1.2 Estrutura do repositório

Mpamba-main/	
├─ mpamba-backend/	
│ └─ api/index.ts	ponto de entrada serverless
│ └─ src/	
│ └─ app.ts	montagem do Express e das rotas
│ └─ server.ts	arranque local (abre porta, agenda timers)
│ └─ config/	prisma, redis, swagger, email
│ └─ middleware/	auth, subscrição/módulo, cache
│ └─ routes/	39 ficheiros de rota (core + module)
│ └─ controller/	adaptação HTTP ↔ serviço
│ └─ services/	50 serviços: regras de negócio
│ └─ shared/	dto, utils, rbac, tipos
│ └─ prisma/	schema, 26 migrações, seeds
│ └─ test/	28 ficheiros de teste
├─ mpamba-frontend/	
│ └─ public/	estáticos (inclui o decreto do PGC)
│ └─ src/	
│ └─ app/(auth)	login, registo, activação, senha
│ └─ app/(modules)	admin, billing, stock, treasury, contabilidade
│ └─ app/core	utilizadores, papéis, subscrição, perfil
│ └─ components/	ui, layout, comuns
│ └─ hooks/	um hook por recurso, sobre TanStack Query
│ └─ services/	chamadas HTTP por recurso
│ └─ shared/	dto (zod), tipos, constantes, utils
│ └─ store/	sessão (Zustand)
└─ docs/	manual do utilizador, este documento, PGC

SIMETRIA PROPOSITADA

`services/`, `hooks/`, `shared/dto/` e `shared/types/` no frontend espelham os recursos do backend. Ao acrescentar um recurso, mexe-se nos mesmos quatro sítios de cada lado.

2. Arquitectura do backend

2.1 Camadas e responsabilidades

O percurso de um pedido é sempre o mesmo, e cada camada tem uma única responsabilidade:

```
HTTP
└─ routes/      declara o caminho e encadeia os guards
    └─ controller/ lê o pedido, valida com o DTO, formata a resposta
        └─ services/ regras de negócio; é aqui que vive a decisão
            └─ prisma      acesso a dados
```

CAMADA	FAZ	NÃO FAZ
Rota	Caminho, método, guards de autenticação, subscrição, módulo e permissão.	Lógica de negócio.
Controlador	<code>schema.parse(req.body)</code> , chama o serviço, devolve <code>{ data }</code> e mapeia o erro para o código HTTP.	Consultar a base de dados directamente.
Serviço	Regras, validações de domínio, transacções, emissão de eventos.	Conhecer <code>req</code> ou <code>res</code> .

Os serviços não recebem o `organizationId` por parâmetro: leem-no do contexto (capítulo 3). É o que permite chamá-los de um controlador, de um handler de evento ou de um teste sem mudar a assinatura.

2.2 Composição das rotas

Cada módulo tem um `index.ts` que aplica os guards comuns uma única vez e monta os sub-routers. É aqui que se garante que nenhuma rota do módulo escapa às verificações:

```
SRC/ROUTES/MODULE/ACCOUNTING/INDEX.TS
router.use(authMiddleware);
router.use(subscriptionGuard);
router.use(moduleGuard('contabilidade'));

router.use('/accounts', accountRoutes);
router.use('/entries', journalRoutes);
router.use('/reports', reportRoutes);
```

Dentro de cada sub-router, a permissão concreta é declarada rota a rota com `permissionGuard(PERMISSIONS.X)`.

ORDEM DAS ROTAS

Rotas literais têm de ser declaradas **antes** das paramétricas. `GET /accounts/pgc` está acima de `GET /accounts/:id`; invertido, o Express leria «pgc» como um id de conta. Vale para qualquer caminho fixo que partilhe o prefixo com um parâmetro.

2.3 Serviços base

Cada módulo tem um `base.service.ts` com os *getters* de contexto, de que os serviços do módulo herdam:

PADRÃO DE SRC/SERVICES/MODULE/*/BASE.SERVICE.TS

```
export abstract class BaseAccountingService {
  protected get orgId(): string {
    const id = organizationContext.getOrganizationId();
    if (!id) throw new Error('Contexto de organização obrigatório para contabilidade');
    return id;
  }
  protected get userId(): string { /* idem */ }
}
```

O *getter* lança se o contexto estiver vazio. Isso transforma um esquecimento de configuração num erro imediato e explícito, em vez de uma consulta silenciosa a todas as organizações.

3. Isolamento entre organizações

Todos os dados de negócio pertencem a uma organização. O isolamento é a propriedade mais importante do sistema: uma falha aqui expõe dados de um cliente a outro.

3.1 O contexto de organização

`shared/utils/organization.context.ts` envolve um `AsyncLocalStorage` do Node. O `authMiddleware` lê o JWT e corre o resto do pedido dentro desse contexto:

```
SRC/MIDDLEWARE/AUTH.MIDDLEWARE.TS

const payload = JwtStrategy.verify(token);
req.user = payload;

organizationContext.run(
  { organizationId: payload.organizationId || '', userId: payload.sub, roles: payload.roles
  || [] },
  () => next()
);
```

Como o `AsyncLocalStorage` acompanha a cadeia de chamadas assíncronas, qualquer serviço invocado dentro do pedido — por mais fundo que esteja — lê o mesmo contexto sem o receber por parâmetro. Pedidos concorrentes não se contaminam: cada um tem o seu *store*.

3.2 Onde o isolamento se aplica

- **Leituras** — todo o `findMany` / `findFirst` de dados de negócio filtra por `organizationId`.
- **Escritas** — o `organizationId` vem do contexto, nunca do corpo do pedido.
- **Acesso por id** — usa-se `findFirst({ where: { id, organizationId } })` e não `findUnique({ where: { id } })`: com o id sozinho, quem adivinhasse um UUID de outra organização acedia-lhe.
- **Referências cruzadas** — ao aceitar um id no corpo (categoria, conta, cliente), confirma-se que pertence à organização antes de o usar.

REGRA AO ESCREVER CÓDIGO NOVO

`findUnique({ where: { id } })` em dados de negócio é quase sempre um erro. A exceção são as entidades globais da plataforma (`Plan`, `Module`, `Permission`), que não pertencem a nenhuma organização.

O Super Admin da plataforma é o único caso sem `organizationId`. Os guards de subscrição e de módulo deixam-no passar por essa ausência, e as rotas de administração operam explicitamente sobre a organização indicada.

4. Autenticação

4.1 Tokens e validade

TOKEN	VALIDADE	NOTAS
Access token	15m	Enviado em <code>Authorization: Bearer</code> . Transporta <code>sub</code> , <code>organizationId</code> e <code>roles</code> .
Refresh token	30m	Segredo próprio (<code>JWT_REFRESH_SECRET</code>). Serve só para obter um access token novo.
Token de senha	1h	<code>PasswordResetToken</code> , persistido e de uso único.

Os prazos vêm de `JWT_EXPIRES_IN` e `JWT_REFRESH_EXPIRES_IN`. As senhas são guardadas com `bcrypt`.

O JWT NÃO É A FONTE DE VERDADE DAS PERMISSÕES

O token traz os papéis, mas o `permissionGuard` consulta a base de dados a cada pedido (`rbacService.hasPermission`). Uma permissão retirada tem efeito imediato, sem esperar a expiração do token — ao custo de uma consulta por pedido protegido.

4.2 Renovação no cliente

O `apiClient` (axios) injecta o token em cada pedido e, ao receber `401`, renova e repete o pedido original uma vez. As renovações concorrentes são **coalescidas** numa só promessa:

```
SRC/SHARED/UTILS/API.UTILS.TS
let refreshPromise: Promise<string> | null = null;

async function refreshAccessToken(): Promise<string> {
  if (!refreshPromise) {
    refreshPromise = (async () => { /* ... */ })()
      .finally(() => { refreshPromise = null; });
  }
  return refreshPromise;
}
```

Sem isto, um ecrã com seis consultas em paralelo dispararia seis renovações ao expirar o token. Falhando a renovação, a sessão é encerrada e o utilizador volta a `/signin`.

4.3 Perfil e sincronização de sessão

`GET /auth/me` devolve o perfil **construído da base de dados** — papéis, permissões e módulos activos — e não o conteúdo do token, que não os traz actualizados. O `SessionSync` no layout raiz refresca o perfil

ao abrir a aplicação e quando a janela ganha foco, para que uma alteração de acessos ou a activação de um módulo apareçam sem novo login.

5. Autorização

Quatro barreiras independentes, sempre nesta ordem. Cada uma responde a uma pergunta diferente e devolve um código próprio.

AS QUATRO BARREIRAS

GUARD	PERGUNTA	HTTP	CÓDIGO
<code>authMiddleware</code>	Quem é?	401	—
<code>subscriptionGuard</code>	A subscrição permite escrever?	402	<code>SUBSCRIPTION_REQUIRED</code>
<code>moduleGuard</code>	O módulo está contratado?	403	<code>MODULE_LOCKED</code>
<code>permissionGuard</code>	O papel autoriza esta acção?	403	<code>FORBIDDEN_PERMISSION</code>

LEITURA CONTINUA, ESCRITA BLOQUEIA

O `subscriptionGuard` só bloqueia `POST`, `PUT`, `PATCH` e `DELETE`. Com a subscrição inactiva, a organização continua a **consultar** os seus dados — o que evita transformar um atraso de pagamento em perda de acesso ao histórico.

5.1 RBAC e perfis

O modelo é `User` → `UserRole` → `Role` → `RolePermission` → `Permission`. Um utilizador pode ter vários papéis e acumula as permissões de todos. As permissões são *strings* com a forma `recurso:acção` ou `domínio:recurso:acção` — por exemplo `invoice:create`, `accounting:entry:create`.

`ROLE_TEMPLATES` define os perfis criados em cada organização nova (`AuthService.register` chama `RoleTemplateService.createRoleTemplatesForOrganization`). O `upsert` sincroniza as permissões do papel, pelo que alterar um template afecta organizações novas — mas **não** as existentes, porque a função só corre no registo.

AO ALTERAR OS TEMPLATES

Acrescentar uma permissão a um `ROLE_TEMPLATE` não a concede a quem já existe. Para isso é preciso uma migração de dados — como a `grant_customer_permissions_to_billing_roles`, que concedeu `customer:*` a todo o papel que já tivesse a permissão de facturação equivalente.

5.2 Super Admin

O `permissionGuard` deixa passar sem verificação quem tenha o papel `SUPER_ADMIN` ou `Super Administrador`. É a única excepção ao RBAC, e existe para a área de administração da plataforma.

6. Modelo de dados

45 modelos Prisma. Quase todos têm `organizationId` ; as exceções são as entidades globais da plataforma.

6.1 Domínios e entidades

DOMÍNIO	MODELOS
Plataforma globais	Plan · PlanModule · Module · Permission
Organização e acesso	Organization · OrganizationModule · Subscription · SubscriptionRequest · ActivationCode · User · UserModule · UserRole · Role · RolePermission · PasswordResetToken
Facturação	Customer · Service · TaxRule · InvoiceSeries · Invoice · InvoiceItem · Proforma · ProformaItem · CreditNote · CreditNoteItem · Receipt
Stock	Product · ProductCategory · Supplier · StockMovement
Tesouraria	FinancialAccount · FinancialCategory · FinancialMovement · CashSession · BankStatement · BankStatementLine · CostCenter · ExchangeRate · Payable · Receivable
Contabilidade	AccountingAccount · JournalEntry · JournalEntryLine
Notificações	Notification · NotificationRecipient

6.2 Enumerações

ENUM	VALORES
SubscriptionStatus	TRIAL · ACTIVE · PAST_DUE · SUSPENDED · EXPIRED · CANCELLED
InvoiceStatus	DRAFT · ISSUED · CANCELLED
PaymentStatus	PENDENTE · PARCIAL · PAGO · VENCIDO
MovementType	ENTRADA · SAIDA · AJUSTE
AccountType	CAIXA · BANCO
FinancialMovementType	ENTRADA · SAIDA · TRANSFERENCIA
AccountSide	ATIVO · PASSIVO · CAPITAL_PROPRIO · CUSTO · PROVEITO
JournalEntrySource	MANUAL · AUTOMATIC

ENUM	VALORES
TaxRuleKind	TAX · RETENTION · EXEMPTION
TaxRuleScope	GLOBAL · CUSTOMER · CUSTOMER_CATEGORY · SERVICE · SERVICE_CATEGORY
NotificationType	INFO · SUCCESS · WARNING · SYSTEM
CashSessionStatus	OPEN · CLOSED
StatementStatus	PENDING · PARTIAL · COMPLETED

6.3 Padrões recorrentes

- **Unicidade por organização** — `@@unique([sku, organizationId])`, `@@unique([code, organizationId])`. O mesmo código pode existir em empresas diferentes.
- **Desactivar em vez de apagar** — `isActive` em quase todas as entidades. Registos com histórico associado nunca são eliminados.
- **Rasto de auditoria** — `userId` em movimentos e lançamentos; `createdAt` / `updatedAt` em toda a parte.
- **Reversão em vez de correcção** — `reversedById` em `JournalEntry`: o original mantém-se e a reversão fica-lhe ligada.
- **Marcas de alerta** — `overdueAlertedAt`, `criticalBalanceAlertedAt`, `expiryAlertedAt` evitam que a verificação diária repita o mesmo aviso.

7. Integração entre módulos

Os módulos não se chamam directamente. Emitem eventos, e o `IntegrationService` é o único sítio que sabe ligar uns aos outros.

7.1 Event bus

`shared/utils/event-bus.ts` é um *singleton* sobre o `EventEmitter` do Node, com o limite de ouvintes elevado a 20 para vários módulos poderem reagir ao mesmo evento. Os nomes estão em `EVENTS`, para não se escreverem à mão.

```
EMISSÃO, EM INVOICE.SERVICE.TS
eventBus.emitEvent(EVENTS.BILLING.INVOICE_ISSUED, issuedInvoice);
```

7.2 Reacções

`IntegrationService.init()` subscreve os handlers. Cada handler verifica se o módulo de destino está activo antes de agir, e envolve cada reacção no seu próprio `try/catch`:

```
SRC/SERVICES/MODULE/INTEGRATION.SERVICE.TS
const isStockActive = await SubscriptionService.isModuleEnabled(orgId, 'stock');
if (isStockActive) await IntegrationService.handleStockReduction(invoice);
```

O isolamento dos `try/catch` é deliberado: uma falha na baixa de stock não pode impedir o lançamento contabilístico da mesma factura, nem vice-versa, e nenhuma delas pode fazer falhar a emissão.

EVENTOS E REACÇÕES

EVENTO	STOCK	TESOURARIA	CONTABILIDADE
<code>invoice.issued</code>	Baixa dos produtos	—	Clientes / Vendas / IVA
<code>invoice.cancelled</code>	Reposição do stock	Estorno da entrada	Reversão do lançamento
<code>receipt.issued</code>	—	Entrada	Caixa ou banco / Clientes
<code>credit-note.issued</code>	—	Saída	Reversão do crédito e do IVA

7.3 Idempotência

Em serverless o mesmo evento pode ser processado mais de uma vez. Cada reacção verifica antes de escrever:

- **Stock** — procura um `StockMovement` com a referência `INV:<número>` antes de dar baixa.

- **Contabilidade** — procura um `JournalEntry` com o mesmo `sourceReference` (`INV:` , `REC:` , `NC:` , `REVERSAL:`).
- **Tesouraria** — usa a referência do movimento pela mesma via.

COLD START EM SERVERLESS

`api/index.ts` chama `IntegrationService.init()` a cada arranque a frio. Sem a guarda `private static initialized` , uma instância reaproveitada acumularia subscrições e o mesmo evento seria processado várias vezes — a mesma factura baixaria stock duas vezes. Ao acrescentar subscrições, mantenha-as dentro dessa guarda.

LIMITE CONHECIDO

O bus é em memória e por instância. Um handler que falhe não é repetido automaticamente, e um documento emitido enquanto o módulo de destino estava inactivo **não** é reprocessado ao activá-lo. Para garantias de entrega seria preciso uma fila persistente.

8. Módulo de Contabilidade

O plano de contas segue o PGC-Angola (Decreto n.º 82/01). O ficheiro `default-accounts.constants.ts` é a fonte única: dele saem a sementeira, a consulta na aplicação e as contas-âncora dos lançamentos automáticos.

8.1 Plano de contas e sincronização

`DEFAULT_ACCOUNTS` tem ~254 contas com apenas `code`, `name` e `side`. A classe e o código-mãe são derivados do código, para não poderem contradizê-lo:

```
accountClassOf('34.5.3') // 3 - primeiro dígito
parentCodeOf('34.5.3')   // '34.5'
parentCodeOf('34')       // undefined - não há contas de 1 dígito no PGC
```

`ensureDefaultAccounts(orgId)` sincroniza conta a conta, e não «só se a organização não tiver nenhuma»: cria as que faltam com `createMany` e repara `class`, `side` e `parentId` agrupando as escrituras em poucos `updateMany`. Não toca em `name` nem em `isActive`, que são decisões da organização.

É chamado uma vez por operação — `listAccounts` no início, e `ensurePlan` no serviço de lançamentos automáticos, antes de resolver as âncoras em paralelo. Chamá-lo de dentro de `getAccountId` punha várias sincronizações a correr ao mesmo tempo.

8.2 Lançamentos automáticos

`ANCHOR_ACCOUNT_CODES` fixa as contas que o sistema movimenta:

ÂNCORA	CÓDIGO	CONTA
CAIXA	45.1	Fundo fixo
BANCOS	43.1	Depósitos à ordem — moeda nacional
CLIENTES	31.1	Clientes — correntes
IVA_LIQUIDADO	34.5.3	IVA liquidado
VENDAS	61.3	Vendas — mercadorias
PRESTACOES_SERVICOS	62.1	Serviços principais

`deleteAccount` recusa eliminar qualquer uma delas. O rédito da factura é repartido pelas linhas com `serviceId` (crédito em 62.1) e pelas restantes (crédito em 61.3), porque o PGC separa vendas de prestações de serviços.

8.3 Ordenação de códigos

NUNCA ORDENAR CÓDIGOS DE CONTA COMO TEXTO

`ORDER BY code` coloca `75.2.11` antes de `75.2.9` e `68.10` antes de `68.9`. Use `compareAccountCodes`, que compara segmento a segmento como número. Existe no backend (`default-accounts.constants.ts`) e espelhado no frontend (`shared/utils/accounting.utils.ts`).

8.4 Validações dos lançamentos manuais

`createManualEntry` recusa: menos de duas linhas; débito e crédito na mesma linha; linha sem valor; lançamento não balanceado; contas de outra organização; contas desactivadas; contas que agregam sub-contas; data futura. A verificação das contas faz duas consultas — uma para as contas em si, outra para saber quais têm filhos (`distinct: ['parentId']`).

9. Convenções da API

Base: `/api`. Grupos montados em `src/app.ts`:

PREFIXO	CONTEÚDO
<code>/auth</code>	Login, registo, activação, refresh, perfil, senha.
<code>/users</code> · <code>/roles</code> · <code>/permissions</code>	RBAC.
<code>/plans</code> · <code>/modules</code>	Catálogo da plataforma.
<code>/organizations</code>	Organizações e a própria (<code>/me</code>).
<code>/subscription</code> · <code>/subscription-requests</code>	Subscrição e pedidos.
<code>/billing</code> · <code>/stock</code> · <code>/treasury</code> · <code>/accounting</code>	Módulos de negócio.
<code>/dashboard</code> · <code>/notifications</code>	Indicadores e notificações.
<code>/cron</code>	Tarefas agendadas, autenticadas por <code>CRON_SECRET</code> .

9.1 Formato das respostas

RECURSO ÚNICO E COLEÇÃO

```
{ "data": { ... } }  
{ "data": [ ... ] }
```

COLEÇÃO PAGINADA

```
{  
  "data": [ ... ],  
  "pagination": { "page": 1, "pageSize": 20, "total": 57, "totalPages": 3,  
                  "hasNextPage": true, "hasPreviousPage": false }  
}
```

ERRO

```
{ "message": "...", "code": "MODULE_LOCKED" }           // code só quando há  
{ "message": "Dados inválidos", "errors": [ ... ] }    // falha de validação zod
```

Parâmetros de listagem: `page`, `pageSize` (20 por defeito), `search` e filtros próprios do recurso. Datas em ISO.

9.2 Códigos de estado

CÓDIGO	QUANDO
400	Validação falhou, ou regra de negócio recusou a operação.
401	Sem token, token inválido ou expirado.
402	Subscrição não permite escrever.
403	Módulo não contratado, ou permissão em falta.
404	Recurso não encontrado na organização em contexto.
500	Erro não previsto.

9.3 Documentação Swagger

Anotações `@swagger` nas rotas alimentam a especificação servida em `/api-docs` (`/docs` redirecciona). A cobertura das anotações é desigual — as rotas mais recentes têm-nas, outras não.

10. Arquitectura do frontend

LER ANTES DE ESCREVER CÓDIGO

O projecto usa Next.js 16 e React 19. Há APIs e convenções que mudaram face a versões anteriores. O `AGENTS.md` do frontend obriga a consultar `node_modules/next/dist/docs/` antes de escrever código novo, em vez de assumir o que se sabia do App Router.

10.1 Grupos de rotas e layouts

- `app/auth/` — login, registo, activação, recuperação de senha. Sem sessão.
- `app/(modules)/` — um directório por módulo (`admin` , `billing` , `stock` , `treasury` , `contabilidade`), cada um com o seu `layout.tsx` .
- `app/core/` — ecrãs comuns: utilizadores, papéis, subscrição, configurações, perfil.

Cada módulo declara a navegação num `*.config.ts` tipado como `LayoutConfig` : logótipo, `routeLabels` (usados no *breadcrumb*) e `groups` com os itens. Cada item traz a `permission` que o torna visível, ou `null` para visível sempre.

```
APP/(MODULES)/CONTABILIDADE/_COMPONENTS/ACCOUNTING.CONFIG.TS
{ name: 'Plano de Contas', href: '/contabilidade/accounts',
  icon: ListTree, permission: PERMISSIONS.ACCOUNTING_ACCOUNT_VIEW }
```

Acrescentar um ecrã ao menu é acrescentar uma linha aqui. A permissão tem de ser a **mesma** que o guard da rota exige no backend — a divergência entre as duas é o defeito mais fácil de introduzir e o mais confuso de diagnosticar (o item aparece e a acção falha com 403).

10.2 Camadas service, hook e DTO

CAMADA	RESPONSABILIDADE
<code>services/</code>	Chamada HTTP e desembrulho do <code>{ data }</code> . Sem estado.
<code>hooks/</code>	<code>useQuery</code> / <code>useMutation</code> , chaves de cache e invalidação.
<code>shared/dto/</code>	Esquemas zod dos formulários, via <code>zodResolver</code> .
<code>shared/types/</code>	Formas devolvidas pela API.

As invalidações de cada módulo estão centralizadas num `hooks/module/*/utils.ts` , para uma mutação não deixar ecrãs vizinhos com dados velhos.

10.3 Estado e cache

- **TanStack Query** — dados do servidor. Chaves em *kebab-case* com o id quando aplicável: `['accounting-account', id]`.
- **Zustand** (`store/auth.store.ts`) — sessão: tokens e perfil. É de onde o `ApiClient` lê o token.
- **Dados imutáveis** — a referência do PGC usa `staleTime: Infinity` : é igual para todas as organizações e não muda entre versões.

11. Validação

Zod nos dois lados, com esquemas de forma equivalente mas propositadamente não partilhados: os projectos são deployados em separado e não têm um pacote comum.

LADO	ONDE	PAPEL
Frontend	<code>shared/dto/*.dto.ts</code>	Feedback imediato no formulário, via <code>zodResolver</code> .
Backend	<code>shared/dto/*.dto.ts</code>	Fronteira de confiança. Valida sempre, ignorando o que o cliente já validou.

Os esquemas convivem com o que os formulários HTML realmente enviam. Dois casos que valem por si:

```
INPUT DE DATA VAZIO DEVOLVE '', NÃO UNDEFINED
date: z.preprocess(
  (value) => (value === '' || value === null ? undefined : value),
  z.coerce.date().optional()
)
```

Sem o `preprocess`, `z.coerce.date()` converte `''` num *Invalid Date* e a validação falha com uma mensagem que não ajuda ninguém.

```
NO FRONTEND, SEM TRANSFORM
const optionalExpiryDate = z.string()
  .refine((v) => v === '' || !Number.isNaN(Date.parse(v)), 'Data de validade inválida')
  .nullable().optional();
```

PORQUE NÃO HÁ TRANSFORM NO FRONTEND

`.transform()` faz o tipo de entrada diferir do de saída, e o `zodResolver` do react-hook-form deixa de casar com o tipo do formulário — o `tsc` falha. As conversões ficam para o backend.

As regras que dependem de mais de um campo usam `.refine()` com `path`, para o erro aparecer no campo certo — por exemplo, a classe de uma conta ter de corresponder ao primeiro dígito do código.

12. Base de dados e migrações

12.1 Prisma e adaptador

Prisma 7 com o adaptador `@prisma/adapters-pg`. O cliente é gerado para `generated/prisma` (fora de `node_modules`) e instanciado uma única vez em `config/prisma.config.ts`. `prisma.config.ts` na raiz aponta o schema, o directório de migrações e a *datasource*.

DEPOIS DE ALTERAR O SCHEMA

Corra `npx prisma generate`. Sem isso o `tsc` falha com «Property X does not exist» em campos que acabou de acrescentar — o cliente gerado ainda não os conhece.

12.2 Migrações de dados

Além das de esquema, há migrações que reescrevem dados. O padrão está em `align_chart_of_accounts_pgc_angola`, que reenumerou o plano de contas de estrutura antiga para a do PGC:

1. Identificar as organizações visadas por um marcador inequívoco — a conta `2432`, porque códigos de 4 dígitos não existem no PGC.
2. Desligar a hierarquia (`parentId = NULL`), que a aplicação reconstrói depois pelo código.
3. Prefixar os códigos a reenumerar com `~`. Sem este passo intermédio, renomear `42 → 11` colidiria com o `11` antigo na *unique* (`code, organizationId`).
4. Renumerar cada conta para o código, nome, classe e lado novos.
5. Limpar o que deixou de fazer sentido, com guarda (`NOT EXISTS` sobre `JournalEntryLine`) para não violar chaves estrangeiras.

Renumerar mantendo a mesma linha — em vez de criar contas novas e apagar as antigas — é o que faz os lançamentos e saldos acompanharem a conta.

COMENTÁRIOS SEM ACENTUAÇÃO NAS MIGRAÇÕES

Os comentários dos ficheiros `.sql` seguem a convenção já existente no repositório e não levam acentos, para evitar problemas de codificação em ferramentas que os leiam com outra *collation*.

12.3 Seeds

`npm run seed` corre `prisma/seed.ts`, que orquestra `prisma/seed/*` por esta ordem: permissões, módulos, planos, utilizadores e organizações, subscrições, e depois dados de exemplo de tesouraria, stock e séries de facturação. É idempotente por `upsert`.

13. Deployment e ambiente

13.1 Projectos

PROJECTO VERCEL	ORIGEM	NOTAS
mpamba-api	CLI, manual	Backend. <code>vercel.json</code> reescreve tudo para <code>/api</code> .
mpamba-erp-jx5d	Git, automático	Frontend em produção. Deploya o <code>main</code> a cada push.
mpamba-erp	—	Projecto duplicado, sem domínio a resolver. É para onde o <code>.vercel</code> local do frontend aponta, o que faz o deploy pelo CLI falhar com <i>Root Directory does not exist</i> .

13.2 Migrações no build

```
MPAMBA-BACKEND/VERCEL.JSON
```

```
"buildCommand": "npx prisma migrate deploy && npx prisma generate"
```

As migrações correm no build, que é o único sítio com acesso ao `DATABASE_URL` de produção — está marcado como *sensitive* no Vercel e o `vercel env pull` devolve `[SENSITIVE]`. Falhando uma migração, o build falha e o deploy anterior continua a servir; o Prisma corre cada migração numa transacção, pelo que a base não fica a meio.

13.3 Variáveis de ambiente

VARIÁVEL	USO
<code>DATABASE_URL</code>	PostgreSQL. Sensitive em produção.
<code>JWT_SECRET</code> · <code>JWT_REFRESH_SECRET</code>	Assinatura dos tokens.
<code>JWT_EXPIRES_IN</code> · <code>JWT_REFRESH_EXPIRES_IN</code>	Validades (<code>15m</code> , <code>30m</code>).
<code>REDIS_HOST</code> · <code>REDIS_PORT</code> · <code>REDIS_PASSWORD</code>	Cache.
<code>EMAIL_HOST</code> · <code>EMAIL_PORT</code> · <code>EMAIL_USER</code> · <code>EMAIL_PASS</code>	SMTP: códigos de activação, recuperação de senha, envio de documentos.
<code>RATE_LIMIT_WINDOW</code> · <code>RATE_LIMIT_MAX</code>	Limitação de pedidos (<code>60 s</code> , <code>100</code>).
<code>CRON_SECRET</code>	Autentica <code>/api/cron/*</code> . Sem ela, as tarefas respondem 503.

VARIÁVEL	USO
NEXT_PUBLIC_API_BASE_URL	Frontend: base da API.

`shared/utils/env.utils.ts` centraliza a leitura com valores por defeito. O `.env.example` serve de referência.

13.4 Tarefas agendadas

MPAMBA-BACKEND/VERCEL.JSON

```
"crons": [ { "path": "/api/cron/alerts", "schedule": "0 6 * * *" } ]
```

Em serverless não há processo persistente, pelo que o `setInterval` de `src/server.ts` nunca corre em produção. As verificações passam por esta rota, protegida por `Authorization: Bearer <CRON_SECRET>`, que o Vercel envia automaticamente. `AlertsService.runChecks()` corre as quatro verificações, cada uma no seu `try/catch`.

13.5 Ordem de deploy

1. Push para `main` — o frontend deploys sozinho.
2. `vercel --prod` em `mpamba-backend` — migra e deploys a API.

JANELA DE INCONSISTÊNCIA

Entre o deploy do frontend e o da API há um intervalo em que a interface nova fala com a API antiga. Ecrãs que dependam de endpoints novos mostram erro nesse período. Numa alteração que mude o contrato entre os dois, deploye a API primeiro.

14. Testes

Vitest, ambiente `node`, sem base de dados: `npm test`. 28 ficheiros, 257 testes. Padrão `test/**/*.test.ts`.

14.1 Padrão de mock

```
vi.mock('.../src/config/prisma.config.js', () => ({
  prisma: { accountingAccount: { findMany: vi.fn() }, journalEntry: { create: vi.fn() } },
}));

vi.mock('.../src/shared/utils/organization.context.js', () => ({
  organizationContext: {
    getOrganizationId: vi.fn().mockReturnValue('org-1'),
    getUserId: vi.fn().mockReturnValue('user-1'),
    isSuperAdmin: vi.fn().mockReturnValue(false),
    getStore: vi.fn(),
  },
}));
```

Só se declaram os modelos e métodos que o teste usa. O contexto é mockado porque os serviços o leem em vez de o receber.

14.2 O que vale a pena testar

- **DTOs** — com o que o formulário realmente envia, campos vazios incluídos.
- **Regras de recusa** — um teste por motivo de rejeição, e não um teste do «caminho feliz».
- **Integridade de constantes** — `chart-of-accounts.test.ts` verifica que o plano não tem códigos repetidos, que cada sub-conta tem mãe declarada antes de si, e que as âncoras existem e não agregam sub-contas. Apanha erros de edição no ficheiro de constantes.
- **Funções puras** — comparadores e cálculos de datas, com referência fixa em vez de `new Date()`.

UUIDS NOS TESTES

O `z.string().uuid()` do zod 4 valida a versão e a variante. `'11111111-1111-1111-1111-111111111111'` é recusado; use `'11111111-1111-4111-8111-111111111111'`.

15. Armadilhas conhecidas

ARMADILHA	O QUE FAZER
Permissão do menu $\neq$ permissão do guard	O item aparece e a acção devolve 403. Foi o que aconteceu com os clientes: o menu pedia <code>customer:view</code> e as rotas exigiam <code>invoice:view</code> . Ao criar um ecrã, confirme que a constante é a mesma dos dois lados.
Rota literal depois da paramétrica	<code>/pgc</code> tem de vir antes de <code>/:id</code> , senão é lido como um id.
<code>findUnique({ where: { id } })</code>	Fura o isolamento entre organizações. Use <code>findFirst</code> com <code>organizationId</code> .
Ordenar códigos de conta como texto	<code>ORDER BY code</code> desordena a hierarquia. Use <code>compareAccountCodes</code> .
Esquecer <code>prisma generate</code>	O <code>tsc</code> falha em campos novos do schema.
Subscrever eventos fora da guarda de <code>init()</code>	Instâncias reaproveitadas acumulam ouvintes e processam o mesmo evento várias vezes.
Alterar <code>ROLE_TEMPLATES</code> e esperar efeito retroactivo	Só afecta organizações novas. Para as existentes, escreva uma migração de dados.
Numeração por <code>count + 1</code>	<code>JournalEntry.number</code> e outros contadores usam <code>count(...) + 1</code> sem índice único, pelo que podem duplicar sob concorrência. Não corrigido; a correcção passa por índice único mais repetição em caso de conflito.
Constantes de permissão sem linha na base de dados	14 constantes (<code>user_role:*</code> , <code>plan_module:*</code> , <code>organization_module:*</code> , <code>role_permission:create delete</code>) não estão em <code>PERMISSIONS_LIST</code> . Nenhuma está em uso, mas um <code>permissionGuard</code> sobre elas falharia sempre. Semeie a permissão antes de a usar.
Permissões semeadas sem constante	<code>billing:service:*</code> e <code>billing:setting:update</code> existem na base de dados mas não em <code>PERMISSIONS</code> . As rotas de serviços usam hoje <code>invoice:create</code> .
Módulo activado depois do documento emitido	A integração não é retroactiva. Registe os movimentos ou lançamentos de abertura.
Validade de produto por lote	<code>Product.expiryDate</code> é um único campo. Vários lotes com validades diferentes exigiriam uma tabela de lotes e saídas por FEFO.

16. Runbooks

16.1 Ambiente local

```
# base de dados e cache
cd mpamba-backend
npm install
npm run docker:up          # Postgres + Redis via docker-compose
npm run db:migrate         # aplica as migrações
npm run seed               # dados iniciais (opcional)
npm run dev                # http://localhost:4000 · docs em /docs

# aplicação web
cd ../mpamba-frontend
npm install
npm run dev                # http://localhost:3000
```

O frontend precisa de `NEXT_PUBLIC_API_BASE_URL` em `.env.local`, apontado a `http://localhost:4000/api`.

16.2 Deploy para produção

```
git push origin main          # frontend deploya sozinho
cd mpamba-backend && npx vercel --prod --yes # migra e deploya a API

# confirmar que as migrações correram
npx vercel inspect --logs <url-do-deploy> | grep -i migrat
```

16.3 Acrescentar uma permissão

1. `prisma/seed/permissions.ts` — acrescentar a `PERMISSIONS_LIST`.
2. `src/shared/utils/rbac/permission.constants.ts` — acrescentar a constante.
3. `src/shared/utils/rbac/role-templates.constants.ts` — atribuir aos perfis que a devem ter.
4. `mpamba-frontend/src/shared/constants/permission.constants.ts` — espelhar.
5. Aplicar no guard da rota e no item de menu, com a **mesma** constante.
6. Migração de dados, se organizações existentes a devem receber.

16.4 Acrescentar um módulo de negócio

1. `prisma/seed/modules.ts` e `plans.ts` — registar o módulo e associá-lo aos planos.
2. `src/routes/module/<novo>/index.ts` — `authMiddleware`, `subscriptionGuard`, `moduleGuard('<código>')`.
3. `base.service.ts` do módulo, com os *getters* de contexto.

4. Montar em `src/app.ts` .
5. Frontend: `app/(modules)/<novo>/` com `layout.tsx` e `_components/<novo>.config.ts` .
6. Reacções a eventos, se houver, em `IntegrationService.init()` — dentro da guarda de inicialização.

16.5 Regenerar a documentação

```
node docs/build-manual-pdf.mjs                # manual do utilizador
node docs/build-manual-pdf.mjs docs/documentacao-tecnica.html \
                                           docs/Documentacao-Tecnica-Mpamba.pdf
```

O script usa o Chrome instalado, conduzido pelo DevTools Protocol. Não instala dependências.